

```
import sqlite3
```

```
class Database:
```

```
    def __init__(self):
```

```
        self.connection = sqlite3.connect("interview_tracker.db")
```

```
        self.cursor = self.connection.cursor()
```

```
        self.create_tables()
```

```
    def create_tables(self):
```

```
        # Users Table
```

```
        self.cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS users (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    username TEXT UNIQUE,
```

```
    password TEXT
```

```
)
```

```
""")
```

```
        # Topics Table
```

```
        self.cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS topics (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    topic_name TEXT,
```

```
    subject TEXT,
```

```
    priority TEXT,
```

```
    status TEXT
```

```
)
```

```
""")
```

```
        self.connection.commit()
```

```
        # Company Topics Table
```

```
self.cursor.execute("""
CREATE TABLE IF NOT EXISTS company_topics (
id INTEGER PRIMARY KEY AUTOINCREMENT,
company TEXT,
topic TEXT,
status TEXT
)
""")
```

Register User

```
def register_user(self, username, password):
    try:
        self.cursor.execute(
            "INSERT INTO users (username, password) VALUES (?, ?)",
            (username, password)
        )
        self.connection.commit()
        return True

    except sqlite3.IntegrityError:
        return False
```

Login User

```
def login_user(self, username, password):

    self.cursor.execute(
        "SELECT * FROM users WHERE username=? AND password=?",
        (username, password)
    )

    return self.cursor.fetchone()
```

Add Topic

```

def add_topic(self, topic_name, subject, priority, status):

    self.cursor.execute("""
INSERT INTO topics (topic_name, subject, priority, status)
VALUES (?, ?, ?, ?)
""", (topic_name, subject, priority, status))

    self.connection.commit()

# Get All Topics
def get_topics(self):

    self.cursor.execute("SELECT * FROM topics")

    return self.cursor.fetchall()

# Delete Topic
def delete_topic(self, topic_id):

    self.cursor.execute(
        "DELETE FROM topics WHERE id=?",
        (topic_id,)
    )

    self.connection.commit()

# Add Study Log
def add_study_log(self, topic, hours, notes):

    self.cursor.execute("""
INSERT INTO study_logs (topic, hours, notes)
VALUES (?, ?, ?)
""", (topic, hours, notes))

```

```
self.connection.commit()
```

```
# Get Study Logs
```

```
def get_study_logs(self):
```

```
    self.cursor.execute(  
        "SELECT * FROM study_logs"  
    )
```

```
    return self.cursor.fetchall()
```

```
def create_tables(self):
```

```
    # Study Logs Table
```

```
    self.cursor.execute("""
```

```
    CREATE TABLE IF NOT EXISTS study_logs (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        topic TEXT,  
        hours TEXT,  
        notes TEXT
```

```
    )  
    """)
```

```
    # Add Company Topic
```

```
def add_company_topic(self, company, topic, status):
```

```
    self.cursor.execute("""  
    INSERT INTO company_topics (company, topic, status)  
    VALUES (?, ?, ?)  
    """, (company, topic, status))
```

```
    self.connection.commit()
```

Get Company Topics

```
def get_company_topics(self):
```

```
    self.cursor.execute(
        "SELECT * FROM company_topics"
    )
```

```
    return self.cursor.fetchall()
```

Mock Interviews Table

```
self.cursor.execute("""
    CREATE TABLE IF NOT EXISTS mock_interviews (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        technical TEXT,
        communication TEXT,
        confidence TEXT,
        notes TEXT
    )
""")
```

Add Mock Interview

```
def add_mock_interview(
```

```
    self,
    technical,
    communication,
    confidence,
    notes
```

```
):
```

```
    self.cursor.execute("""
        INSERT INTO mock_interviews
        (technical, communication, confidence, notes)
        VALUES (?, ?, ?, ?)
    """, (
```

technical,
communication,
confidence,
notes
)

self.connection.commit()

Get Mock Interviews

def get_mock_interviews(self):

self.cursor.execute(
 "SELECT * FROM mock_interviews"
)

return self.cursor.fetchall()